

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники**

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
дисциплины «Искусственный интеллект и машинное обучение»**

Выполнила:

Федорова Дарья Юрьевна 2 курс,
группа ИВТ-б-о-24-1, 09.03.01
«Информатика и вычислительная
техника», направленность
(профиль) «Программное
обеспечение средств
вычислительной техники и
автоматизированных систем»,
очная форма обучения

(подпись)

Руководитель практики:
Воронкин Р. А.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Вариант 25

Тема: Основные этапы исследовательского анализа данных

Цель: научиться применять методы обработки данных в `pandas.DataFrame`, необходимые для разведочного анализа данных (EDA), включая работу с пропусками, выбросами, масштабирование и кодирование категориальных признаков.

Ссылка на репозиторий: https://github.com/Vishhh123/Laborat_6

Ход работы:

Был создан общедоступный репозиторий с лицензией MIT, языком программирования Python.

```
PS C:\Users\user> git clone https://github.com/Vishhh123/Laborat_6.git
Cloning into 'Laborat_6'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
PS C:\Users\user> cd Laborat_6
```

Рис. 1 - клонирование репозитория на устройство, переход к репозиторию.

Ход работы:

```
: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", None, "Мария"],
    "Возраст": [25, 30, None, 40, 35],
    "Город": ["Москва", None, "Казань", "Новосибирск", "СПб"]
}

df = pd.DataFrame(data)

print(df)

Имя  Возраст  Город
0  Анна   25.0  Москва
1  Иван   30.0   NaN
2  Ольга  NaN   Казань
3  NaN   40.0  Новосибирск
4  Мария  35.0   СПб

: print(df.isna().sum())

Имя  1
Возраст  1
Город  1
dtype: int64

: print(df.notna())

Имя  Возраст  Город
0  Правда  Правда  Правда
1  Правда  Правда  Ложь
2  Правда  Ложь   Правда
3  Ложь   Правда  Правда
4  Правда  Правда  Правда

: pip install missingno

Сбор недостающих данных
Загрузка missingno-0.5.2-py3-none-any.whl.metadata (639 байт)
Требование уже выполнено: numpy в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (из missingno) (2.4.4)
Требование уже выполнено: matplotlib в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (or missingno) (3.11.0)
Требование уже выполнено: scipy в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (or missingno) (1.17.1)
Требование уже выполнено: seaborn в C:\Users\user\anaconda3\Lib\site-packages (or missingno) (0.13.2)
Требование уже выполнено: contourpy>=1.0.1 в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (or matplotlib->missingno) (1.3.3)
Требование уже выполнено: cycycler>=0.10 в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (из matplotlib->missingno) (0.12.1)
Требование уже выполнено: fonttools>=4.22.0 в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (из matplotlib->missingno) (4.62.1)
Требование уже выполнено: kiwisolver>=1.3.1 в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (из matplotlib->missingno) (1.5.0)
```

Рис. 2 – Создание DataFrame с пропусками, проверка пропусков через `.isna().sum()` и `.notna()`.

```
import pandas as pd
import numpy as np
```

```
np.random.seed(42)
```

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий", "Елена", "Сергей", "Алина", "Артем"],
    "Возраст": [25, np.nan, 22, 40, 35, np.nan, 28, 31, np.nan, 29],
    "Город": ["Москва", "СПб", np.nan, "Новосибирск", "СПб", "Екатеринбург", np.nan, "Казань", "Томск", np.nan],
    "Доход": [50000, 60000, np.nan, 70000, 65000, 55000, 48000, np.nan, 52000, 58000],
    "Образование": [np.nan, "Высшее", "Среднее", "Высшее", np.nan, "Высшее", "Среднее", "Высшее", np.nan, "Среднее"],
    "Стаж работы": [3, 8, 1, 15, 10, np.nan, 5, 7, np.nan, 2]
}
```

```
df = pd.DataFrame(data)
print(df)
```

```
Имя  Возраст  Город  Доход  Образование  Стаж работы
0  Анна  25.0  Москва  50000.0  NaN  3.0
1  Иван  NaN  СПб  60000.0  Высшее  8.0
2  Ольга  22.0  NaN  NaN  Среднее  1.0
3  Петр  40.0  Новосибирск  70000.0  Высшее  15.0
4  Мария  35.0  СПб  65000.0  NaN  10.0
5  Дмитрий  NaN  Екатеринбург  55000.0  Высшее  NaN
6  Елена  28.0  NaN  48000.0  Среднее  5.0
7  Сергей  31.0  Казань  NaN  Высшее  7.0
8  Алина  NaN  Томск  52000.0  NaN  NaN
9  Артем  29.0  NaN  58000.0  Среднее  2.0
```

```
import missingno as msno
import matplotlib.pyplot as plt
```

```
msno.matrix(df)
plt.show()
```

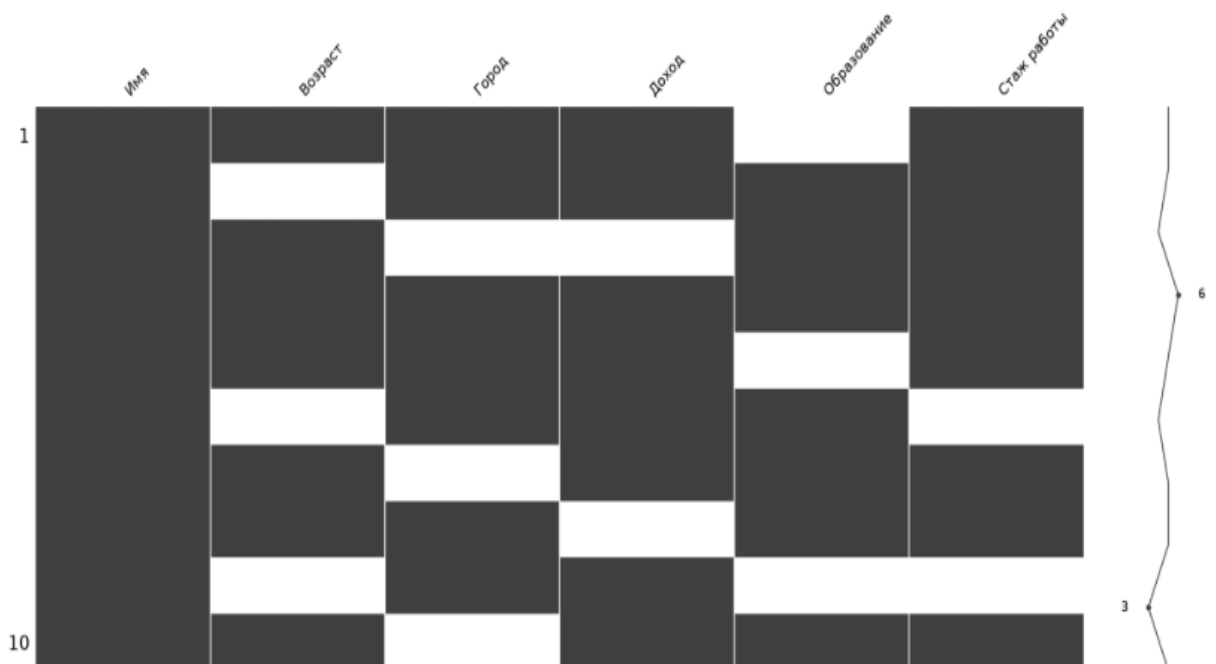


Рис. 3 – Визуализация пропусков с помощью `msno.matrix()`: белые полосы показывают пропуски (NaN) в данных.

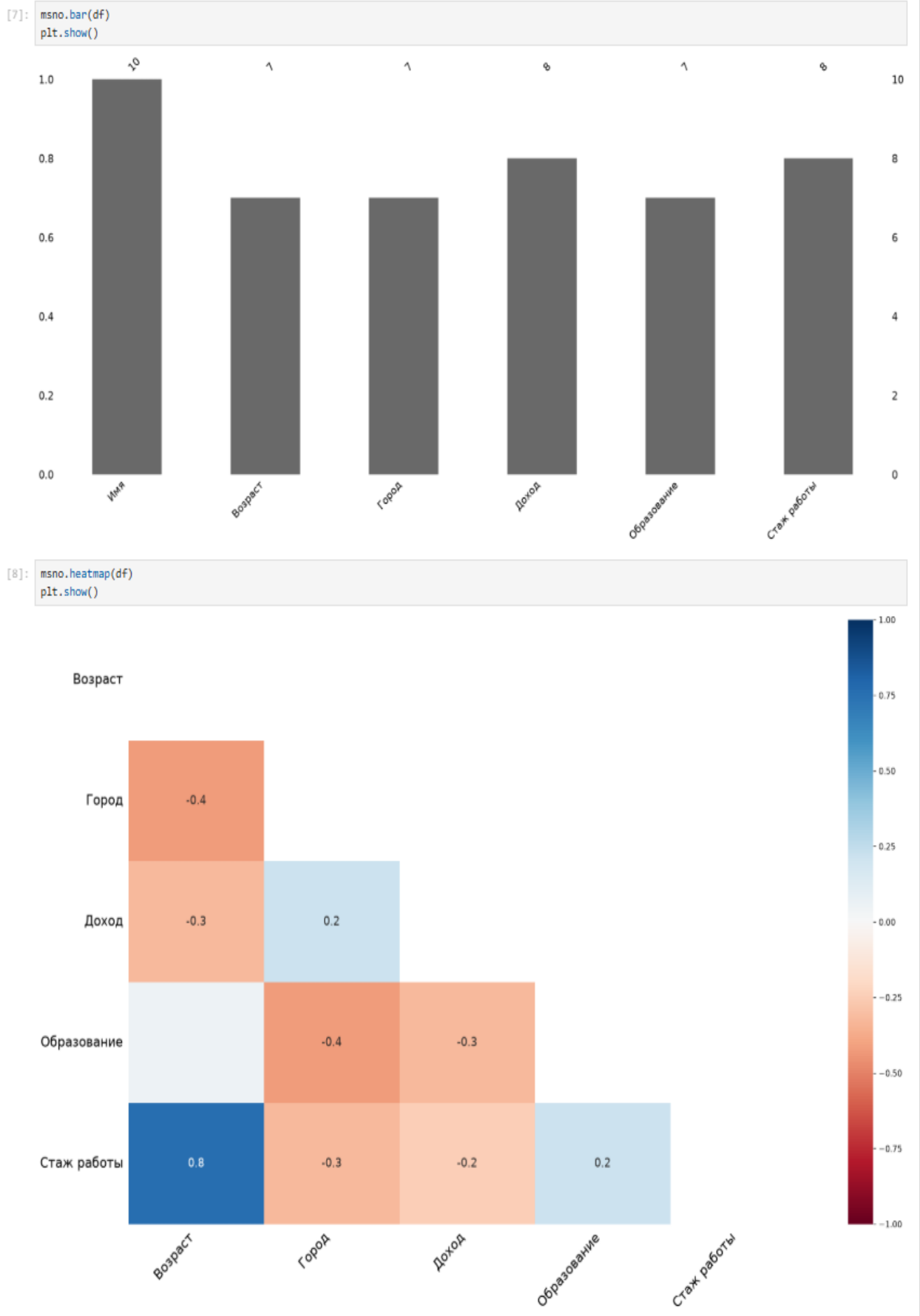


Рис. 4 — Визуализация пропусков: `msno.bar()` показывает заполненность столбцов, `msno.heatmap()` — корреляцию между пропусками.

```
|: import pandas as pd
import numpy as np

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий"],
    "Возраст": [25, np.nan, 22, 40, 35, np.nan],
    "Город": ["Москва", "СПб", np.nan, "Новосибирск", "СПб", "Екатеринбург"],
    "Доход": [50000, 60000, np.nan, 70000, 65000, 55000]
}

df = pd.DataFrame(data)
print(df)
```

```
Имя Возраст Город Доход
0 Анна 25.0 Москва 50000.0
1 Иван NaN СПб 60000.0
2 Ольга 22.0 NaN NaN
3 Петр 40.0 Новосибирск 70000.0
4 Мария 35.0 СПб 65000.0
5 Дмитрий NaN Екатеринбург 55000.0
```

```
|: df_cleaned = df.dropna()
print(df_cleaned)
```

```
Имя Возраст Город Доход
0 Анна 25.0 Москва 50000.0
3 Петр 40.0 Новосибирск 70000.0
4 Мария 35.0 СПб 65000.0
```

```
|: df_cleaned = df.dropna(how="all")
print(df_cleaned)
```

```
Имя Возраст Город Доход
0 Анна 25.0 Москва 50000.0
1 Иван NaN СПб 60000.0
2 Ольга 22.0 NaN NaN
3 Петр 40.0 Новосибирск 70000.0
4 Мария 35.0 СПб 65000.0
5 Дмитрий NaN Екатеринбург 55000.0
```

```
|: df_cleaned = df.dropna(axis=1)
print(df_cleaned)
```

```
Имя
0 Анна
1 Иван
2 Ольга
3 Петр
4 Мария
5 Дмитрий
```

```
|: df_cleaned = df.dropna(thresh=3)
print(df_cleaned)
```

```
Имя Возраст Город Доход
0 Анна 25.0 Москва 50000.0
1 Иван NaN СПб 60000.0
3 Петр 40.0 Новосибирск 70000.0
4 Мария 35.0 СПб 65000.0
5 Дмитрий NaN Екатеринбург 55000.0
```

Рис. 5 - Удаление пропусков: `.dropna()` удаляет строки с NaN, `axis=1` удаляет столбцы, `thresh=3` оставляет строки с минимум 3 непустыми значениями.

```
|: import pandas as pd
import numpy as np

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий"],
    "Возраст": [25, np.nan, 22, 40, 35, np.nan],
    "Город": ["Москва", "СПб", np.nan, "Новосибирск", "СПб", "Екатеринбург"],
    "Доход": [50000, 60000, np.nan, 70000, 65000, 55000]
}

df = pd.DataFrame(data)
print(df)
```

```
Имя  Возраст  Город  Доход
0 Анна   25.0  Москва  50000.0
1 Иван   NaN   СПб     60000.0
2 Ольга  22.0  NaN     NaN
3 Петр   40.0  Новосибирск  70000.0
4 Мария  35.0  СПб     65000.0
5 Дмитрий NaN   Екатеринбург  55000.0
```

```
|: df_cleaned = df.dropna()
print(df_cleaned)
```

```
Имя  Возраст  Город  Доход
0 Анна   25.0  Москва  50000.0
3 Петр   40.0  Новосибирск  70000.0
4 Мария  35.0  СПб     65000.0
```

```
|: df_cleaned = df.dropna(how="all")
print(df_cleaned)
```

```
Имя  Возраст  Город  Доход
0 Анна   25.0  Москва  50000.0
1 Иван   NaN   СПб     60000.0
2 Ольга  22.0  NaN     NaN
3 Петр   40.0  Новосибирск  70000.0
4 Мария  35.0  СПб     65000.0
5 Дмитрий NaN   Екатеринбург  55000.0
```

```
|: df_cleaned = df.dropna(axis=1)
print(df_cleaned)
```

```
Имя
0 Анна
1 Иван
2 Ольга
3 Петр
4 Мария
5 Дмитрий
```

```
|: df_cleaned = df.dropna(thresh=3)
print(df_cleaned)
```

```
Имя  Возраст  Город  Доход
0 Анна   25.0  Москва  50000.0
1 Иван   NaN   СПб     60000.0
3 Петр   40.0  Новосибирск  70000.0
4 Мария  35.0  СПб     65000.0
5 Дмитрий NaN   Екатеринбург  55000.0
```

Рис. 6 – Применение `.dropna()` с параметрами: удаление строк с любым NaN, удаление только полностью пустых строк, удаление столбцов с пропусками, фильтрация по порогу `thresh=3`.

```

: df_cleaned = df.dropna(axis=1, thresh=4)
: print(df_cleaned)

Имя Возраст Город Доход
0 Анна 25.0 Москва 50000.0
1 Иван NaN СПб 60000.0
2 Ольга 22.0 NaN NaN
3 Петр 40.0 Новосибирск 70000.0
4 Мария 35.0 СПб 65000.0
5 Дмитрий NaN Екатеринбург 55000.0

: df["Возраст"] = df["Возраст"].fillna(30)

: mean_value = df["Доход"].mean()
: df["Доход"] = df["Доход"].fillna(mean_value)

: median_value = df["Возраст"].median()
: df["Возраст"] = df["Возраст"].fillna(median_value)

: mode_value = df["Город"].mode()[0]
: df["Город"] = df["Город"].fillna(mode_value)

: df["Температура"] = df["Температура"].ffill()

: df.ffill(inplace=True)

:
  День  Температура  Температура_interp  Температура_poly
0      1           20.0           20.000000           20.000000
1      2           20.0           21.333333           21.500000
2      3           20.0           22.666667           22.833333
3      4           24.0           24.000000           24.000000
4      5           25.0           25.000000           25.000000

: print(df.columns)

Index(['День', 'Температура', 'Температура_интерп', 'Температура_поли'], dtype='str')

: df.ffill(inplace=True)

:
  День  Температура  Температура_interp  Температура_poly
0      1           20.0           20.000000           20.000000
1      2           20.0           21.333333           21.500000
2      3           20.0           22.666667           22.833333
3      4           24.0           24.000000           24.000000
4      5           25.0           25.000000           25.000000

```

Рис. 7 – Заполнение пропусков: константой (30), средним, медианой, модой, а также применение ffill() и интерполяции для температурных данных.

```
] pip install scipy
```

Требование уже выполнено: scipy в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (1.17.1)
Требование уже выполнено: numpy<2.7,>=1.26.4 в C:\Users\user\AppData\Roaming\Python\Python313\site-packages (из scipy) (2.4.4)
Примечание: для использования обновленных пакетов может потребоваться перезапустить ядро.

```
] import pandas as pd
import numpy as np

df = pd.DataFrame({
    "День": [1, 2, 3, 4, 5],
    "Температура": [20.0, np.nan, np.nan, 24.0, 25.0]
})

df["Температура_интерп"] = df["Температура"].interpolate()
print(df)
```

```
   День  Температура  Температура_интерп
0  1  20.0  20.000000
1  2   NaN  21.333333
2  3   NaN  22.666667
3  4  24.0  24.000000
4  5  25.0  25.000000
```

```
] df["Температура_poly"] = df["Температура"].interpolate(method="polynomial", order=2)
```

```
] import pandas as pd
import numpy as np

dates = pd.date_range("2024-01-01", periods=5, freq="D")
df = pd.DataFrame({
    "дата": dates,
    "уровень воды": [1.2, np.nan, np.nan, 1.8, 2.0]
})
df.set_index("дата", inplace=True)
df["интерполяция"] = df["уровень воды"].interpolate(method="time")
print(df)
```

```
Интерполяция уровня воды
дата
2024-01-01  1.2  1.2
2024-01-02   NaN  1.4
2024-01-03   NaN  1.6
2024-01-04  1.8  1.8
2024-01-05  2.0  2.0
```

```
] df["Возраст"] = df["Возраст"].interpolate(limit=1, limit_direction="forward")
print(df)
```

```
Имя  Возраст  Зарплата
0  Анна    25  50000
1  Иван    30  60000
2  Ольга   22  45000
3  Петр    40  70000
4  Мария   35  65000
```

Рис. 8 – Интерполяция пропусков: линейная (`interpolate()`), полиномиальная (`method='polynomial'`), с учётом времени (`method='time'`) и с ограничением `limit=1`.


```
df.interpolate(method="linear", axis=0, inplace=True)
```

уровень воды интерполяция		
дата		
2024-01-01	1.2	1.2
2024-01-02	1.4	1.4
2024-01-03	1.6	1.6
2024-01-04	1.8	1.8
2024-01-05	2.0	2.0

```
mean = df["Возраст"].mean()
std = df["Возраст"].std()

z = (df["Возраст"] - mean) / std
outliers = df[abs(z) > 3]

print(outliers)
```

Пустой DataFrame
Столбцы: [Имя, Возраст, Зарплата]
Индекс: []

```
Q1 = df["Возраст"].quantile(0.25)
Q3 = df["Возраст"].quantile(0.75)
IQR = Q3 - Q1

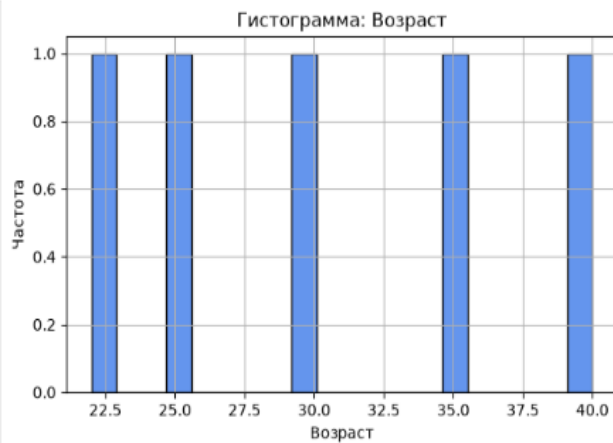
outliers = df[(df["Возраст"] < Q1 - 1.5 * IQR) | (df["Возраст"] > Q3 + 1.5 * IQR)]
print(outliers)
```

Пустой DataFrame
Столбцы: [Имя, Возраст, Зарплата]
Индекс: []

Рис. 9 – Интерполяция методом linear для всего DataFrame, а также обнаружение выбросов методами 3 сигм (z-score) и межквартильного размаха (IQR) — выбросы не обнаружены.

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(6, 4))
plt.hist(df["Возраст"], bins=20, color='cornflowerblue', edgecolor='black')
plt.title("Гистограмма: Возраст")
plt.xlabel("Возраст")
plt.ylabel("Частота")
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(4, 6))
plt.boxplot(df["Возраст"], orientation='vertical')
plt.title("Boxplot: Возраст")
plt.ylabel("Возраст")
plt.grid(True)
plt.tight_layout()
plt.show()
```

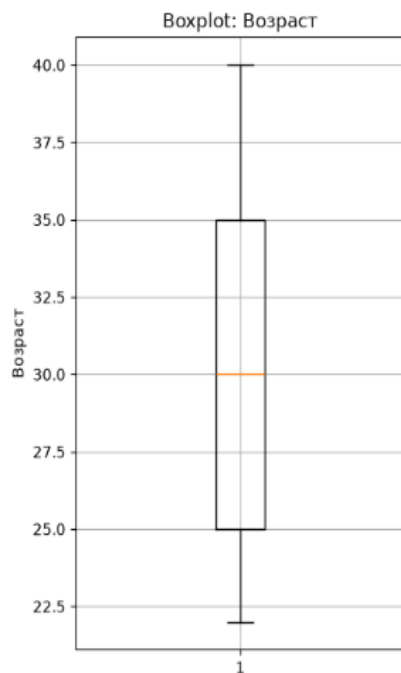


Рис. 10 –Визуализация распределения признака: гистограмма (plt.hist()) и ящик с усами (plt.boxplot()) для обнаружения выбросов.

```

: print(df.columns)

Index(['Имя', 'Возраст', 'Зарплата'], dtype='str')

: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Создаём DataFrame заново
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Занимата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)

print("Столбцы:", df.columns.tolist())

# Строим график
plt.figure(figsize=(7, 5))
plt.scatter(df["Возраст"], df["Занимата"], alpha=0.75, color='seagreen')
plt.title("Scatterplot: Возраст vs Занимата")
plt.xlabel("Возраст")
plt.ylabel("Занимата")
plt.grid(True)
plt.tight_layout()
plt.show()

```

Столбцы: ['Имя', 'Возраст', 'Род занятий']

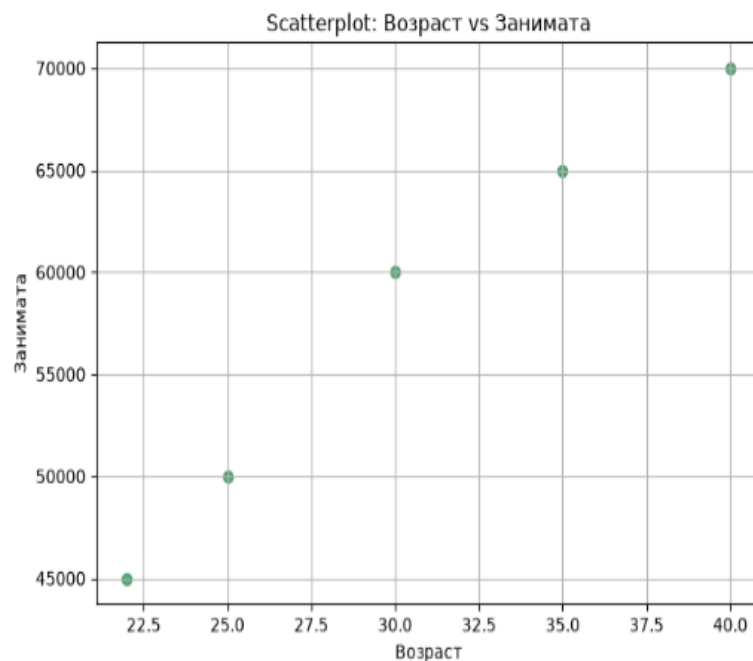


Рис. 11 – Точечный график (scatterplot) для визуализации зависимости между возрастом и доходом, позволяющий обнаружить мультипризнаковые выбросы.

```

: import pandas as pd
import numpy as np

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий", "Елена"],
    "Баланс на счете": [50000, 60000, 45000, 70000, 65000, 400000, 450000]
}

df = pd.DataFrame(data)
print(df)

Имя Остаток на счете
0 Анна 50000
1 Иван 60000
2 Ольга 45000
3 Петр 70000
4 Мария 65000
5 Дмитрий 400000
6 Елена 450000

: Q1 = df["Баланс на счете"].quantile(0.25)
Q3 = df["Баланс на счете"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_removed = df[(df["Баланс на счете"] >= lower) & (df["Баланс на счете"] <= upper)]
print(df_removed)

Имя Остаток на счете
0 Анна 50000
1 Иван 60000
2 Ольга 45000
3 Петр 70000
4 Мария 65000
5 Дмитрий 400000
6 Елена 450000

: df_clipped = df.copy()
df_clipped["Баланс на счете"] = df["Баланс на счете"].clip(lower, upper)
print(df_clipped)

Имя Остаток на счете
0 Анна 50000
1 Иван 60000
2 Ольга 45000
3 Петр 70000
4 Мария 65000
5 Дмитрий 400000
6 Елена 450000

: median = df["Баланс на счете"].median()

df_replaced = df.copy()
df_replaced["Баланс на счете"] = df["Баланс на счете"].apply(
    lambda x: median if x < lower or x > upper else x
)

print(df_replaced)

Имя Остаток на счете
0 Анна 50000
1 Иван 60000
2 Ольга 45000
3 Петр 70000
4 Мария 65000
5 Дмитрий 400000
6 Елена 450000

```

Рис. 12 – Обработка выбросов: удаление по IQR, обрезка через .clip() и замена на медиану.

```

[: df_log = df.copy()
df_log["Лог баланса"] = np.log1p(df["Баланс на счете"])
print(df_log[["Имя", "Баланс на счете", "Лог баланса"]])

Имя Остаток на счете Лог баланса
0 Анна 50000 10.819798
1 Иван 60000 11.002117
2 Ольга 45000 10.714440
3 Петр 70000 11.156265
4 Мария 65000 11.082158
5 Дмитрий 400000 12.899222
6 Елена 450000 13.017005

[: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)

Имя Возраст Зарплата
0 Анна 25 50000
1 Иван 30 60000
2 Ольга 22 45000
3 Петр 40 70000
4 Мария 35 65000

[: df_standardized = df.copy()
df_standardized["Возраст"] = (df["Возраст"] - df["Возраст"].mean()) / df["Возраст"].std()
df_standardized["Зарплата"] = (df["Зарплата"] - df["Зарплата"].mean()) / df["Зарплата"].std()
print(df_standardized)

Имя Возраст Зарплата
0 Анна -0,739657 -0,771589
1 Иван -0,054789 0,192897
2 Ольга -1,150577 -1,253831
3 Петр 1,314945 1,157383
4 Мария 0,630078 0,675140

[: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_values = scaler.fit_transform(df[["Возраст", "Зарплата"]])

df_scaled = df.copy()
df_scaled[["Возраст", "Зарплата"]] = scaled_values
print(df_scaled)

Имя Возраст Зарплата
0 Анна -0,826961 -0,862662
1 Иван -0,061256 0,215666
2 Ольга -1,286384 -1,401826
3 Петр 1,470153 1,293993
4 Мария 0,704448 0,754829

```

Рис. 13 – Логарифмическое преобразование для сжатия выбросов, ручная стандартизация через `.mean()` и `.std()`, а также стандартизация с помощью `StandardScaler`.

```

]: df_robust = df.copy()

for col in ["Возраст", "Зарплата"]:
    median = df[col].median()
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    df_robust[col] = (df[col] - median) / iqr

print(df_robust)

Имя Возраст Зарплата
0 Анна -0,357143 -0,50
1 Иван 0,000000 0,00
2 Ольга -0,571429 -0,75
3 Петр 0,714286 0,50
4 Мария 0,357143 0,25
5 Дмитрий 6,428571 2,00
6 Елена -1,785714 -2,50

]: from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
scaled_values = scaler.fit_transform(df[["Возраст", "Зарплата"]])

df_scaled = df.copy()
df_scaled[["Возраст", "Зарплата"]] = scaled_values
print(df_scaled)

Имя Возраст Зарплата
0 Анна -0,357143 -0,50
1 Иван 0,000000 0,00
2 Ольга -0,571429 -0,75
3 Петр 0,714286 0,50
4 Мария 0,357143 0,25
5 Дмитрий 6,428571 2,00
6 Елена -1,785714 -2,50

]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Образование": ["среднее", "высшее", "начальное", "высшее", "среднее"]
}

df = pd.DataFrame(data)
print(df)

Имя Образование
0 Анна среднее
1 Иван высшее
2 Ольга начальное
3 Петр высшее
4 Мария среднее

]: df_encoded = df.copy()
df_encoded["Образование"] = pd.factorize(df["Образование"])[0]
print(df_encoded)

Имя Образование
0 Анна 0
1 Иван 1
2 Ольга 2
3 Петр 1
4 Мария 0

```

Рис. 14 – Робастное масштабирование вручную и через RobustScaler, а также кодирование категориального признака с помощью pd.factorize().

```
[68]: ordered_mapping = {
    "начальное": 0,
    "среднее": 1,
    "высшее": 2
}

df_ordered = df.copy()
df_ordered["Образование"] = df["Образование"].map(ordered_mapping)
print(df_ordered)

Имя Образование
0 Анна 1
1 Иван 2
2 Ольга 0
3 Петр 2
4 Мария 1

[69]: from sklearn.preprocessing import OrdinalEncoder

encoder = OrdinalEncoder(categories=[["начальное", "среднее", "высшее"]])
X = df[["Образование"]]
X_encoded = encoder.fit_transform(X)

df_encoded = df.copy()
df_encoded["Образование"] = X_encoded
print(df_encoded)

Имя Образование
0 Анна 1.0
1 Иван 2.0
2 Ольга 0.0
3 Петр 2.0
4 Мария 1.0

[70]: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Город": ["Москва", "СПб", "Казань", "Москва", "Казань"]
}

df = pd.DataFrame(data)
print(df)

Имя Город
0 Анна Москва
1 Иван СПб
2 Ольга Казань
3 Петр Москва
4 Мария Казань

[71]: df_encoded = pd.get_dummies(df, columns=["Город"])
print(df_encoded)

Имя Город_Казань Город_Москва Город_СПб
0 Анна Ложно Верно Ложно
1 Иван Ложно Ложно Верно
2 Ольга Верно Ложно Ложно
3 Петр Ложно Верно Ложно
4 Мария Верно Ложно Ложно

[72]: from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[["Город"]])

df_encoded = pd.DataFrame(encoded, columns=encoder.get_feature_names_out(["Город"]))
print(df_encoded)

Город_Казань Город_Москва Город_СПб
0 0,0 1,0 0,0
1 0,0 0,0 1,0
2 1,0 0,0 0,0
3 0,0 1,0 0,0
4 1,0 0,0 0,0

[73]: df_encoded = pd.get_dummies(df, columns=["Город"], drop_first=True)

[74]: encoder = OneHotEncoder(drop='first', sparse_output=False)
```

Рис. 15 — Кодирование порядковых признаков через `.map()` и `OrdinalEncoder`, а также one-hot кодирование номинальных признаков через `pd.get_dummies()` и `OneHotEncoder` с исключением дамми-ловушки (`drop_first=True`).

```

: import pandas as pd

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Олег", "Светлана"],
    "Город": ["Москва", "СПб", "Казань", "Москва", "Казань", "СПб", "Казань"],
    "Доход": [50000, 60000, 45000, 70000, 65000, 58000, 47000]
}

df = pd.DataFrame(data)
print(df)

Имя Город Доход
0 Анна Москва 50000
1 Иван СПб 60000
2 Ольга Казань 45000
3 Петр Москва 70000
4 Мария Казань 65000
5 Олег СПб 58000
6 Светлана Казань 47000

: mean_target = df.groupby("Город")["Доход"].mean()
df_encoded = df.copy()
df_encoded["Город"] = df["Город"].map(mean_target)
print(df_encoded)

Имя Город Доход
0 Анна 60000.000000 50000
1 Иван 59000.000000 60000
2 Ольга 52333.333333 45000
3 Петр 60000.000000 70000
4 Мария 52333.333333 65000
5 Олег 59000.000000 58000
6 Светлана 52333.333333 47000

: print(df.columns)

Индекс(['карат', 'огранка', 'четкость', 'глубина', 'таблица', 'x', 'y', 'z', 'color_E',
'color_F', 'color_G', 'color_H', 'color_I', 'color_J'],
dtype='str')

```

Рис. 16 — Target-кодирование: замена категорий в столбце "Город" на средние значения целевого признака "Доход", а также вывод списка столбцов после обработки датасета.


```
from sklearn.preprocessing import TargetEncoder

X = df[["cut"]]
y = df["carat"]

encoder = TargetEncoder()
X_encoded = encoder.fit_transform(X, y)

df_encoded = df.copy()
df_encoded["cut"] = X_encoded
print(df_encoded)

Таблица глубины огранки «карат» x y \
0 1,776201 0,489388 2 2,447919 -0,141333 1,432148 1,378806
1 -0,127819 -0,185339 3 -0,345564 -0,141333 0,074730 0,128209
2 1,109794 0,185236 4 -0,435676 0,352009 1,204330 1,187875
3 -0,580024 0,185236 3 0,645672 1,338693 -0,532787 -0,520956
4 3,013814 0,050061 1 -2,328036 1,338693 2,495301 2,619857
... ..
49995 2,990014 0,193009 2 0,555560 1,832035 2,248498 2,237995
49996 0,110184 0,190558 7 0,735784 0,352009 0,274071 0,233221
49997 0,348186 0,051893 1 -0,345564 2,325377 0,444935 0,529164
49998 1,157395 -0,182682 1 -0,345564 0,352009 1,147375 1,197422
49999 -0,127819 -0,179307 6 -0,345564 -1,128017 0,055745 0,109116

z color_E color_F color_G color_H color_I color_J
0 1,695091 Ложь False True False False False False
1 0,066886 Истина Истина Истина Истина Истина Истина
2 1,142116 Ложь Ложь Истина Истина Истина Истина 3 -0,470728 Истинно Ложно Ложно Ложно Ложно Ложно
4 2,186624 Ложно Ложно Ложно Ложно Истинно
... ..
49995 2,309508 Ложно Ложно Ложно Истинно Ложно
49996 0,328013 Ложно Истинно Ложно Ложно Ложно
49997 0,450897 Ложь Истина False False False False False False
49998 1,126755 Ложь Истина False False False False False
49999 0,051526 Ложь Ложь False False True False

[44380 строк x 14 столбцов]

import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("https://raw.githubusercontent.com/Pratik-Bhujade/Diamond-Dataset/refs/heads/master/DiamondData.csv")
df.head()
```

	карат	вырезать	Цвет	ясность	глубина	таблица	Цена	x	y	z
0	1.50	Справедливый	G	SI1	64.5	57.0	10352.0	7.15	7.09	4.59
1	0.70	Идеальный вариант	E	ПРОТИВ2	61.4	57.0	2274.0	5.72	5.78	3.53
2	1.22	Премиум	G	ПРОТИВ1	61.3	58.0	8779.0	6.91	6.89	4.23
3	0.51	Премиум	E	ПРОТИВ2	62.5	60.0	1590.0	5.08	5.10	3.18
4	2.02	Очень Хорошо	J	SI2	59.2	60.0	11757.0	8.27	8.39	4.91

Рис. 17 – Применение TargetEncoder для кодирования категориального признака "cut" по целевому признаку "carat", а также загрузка и первые строки датасета diamonds.

```
|: df.drop(columns=["price"], inplace=True)
df.head()
```

```
|:
   карат    вырезать  Цвет  ясность  глубина  таблица  x  y  z
0   1.50    Справедливый   G    SI1    64.5    57.0  7.15  7.09  4.59
1   0.70    Идеальный вариант   E  ПРОТИВ2    61.4    57.0  5.72  5.78  3.53
2   1.22        Премиум   G  ПРОТИВ1    61.3    58.0  6.91  6.89  4.23
3   0.51        Премиум   E  ПРОТИВ2    62.5    60.0  5.08  5.10  3.18
4   2.02    Очень Хорошо   J    SI2    59.2    60.0  8.27  8.39  4.91
```

```
|: df.isna().sum()
```

```
|: карат 0
   грань 0
   цвет 0
   проба 0
   глубина 471
   таблица 390
   x 221
   y 333
   z 428
   dtype: int64
```

```
|: import missingno as msno
msno.matrix(df)
```

```
|: <Оси: >
```

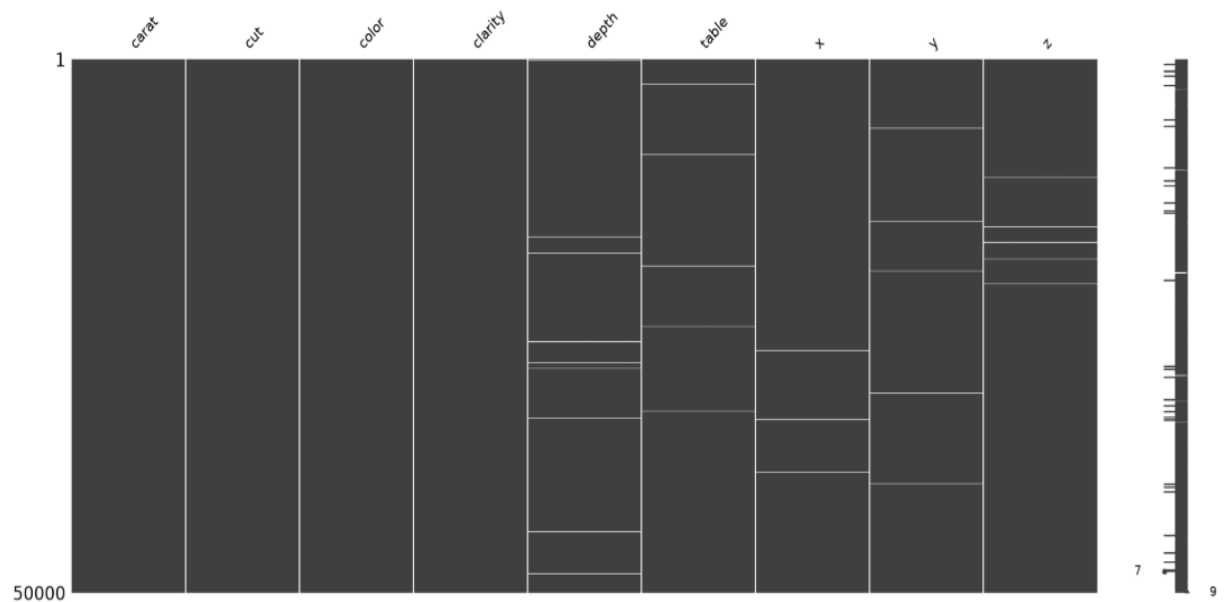


Рис. 18 – Удаление целевого признака price и подсчёт пропусков в каждом столбце датасета diamonds с помощью .isna().sum().

```
df = df.dropna()
df.isna().sum()
```

```
карат 0
огранка 0
цвет 0
прозрачность 0
глубина 0
таблица 0
x 0
y 0
z 0
dtype: int64
```

```
import seaborn as sns

numeric_cols = df.select_dtypes(include='number').columns

for col in numeric_cols:
    plt.figure(figsize=(10, 4))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot: {col}")
    plt.show()
```

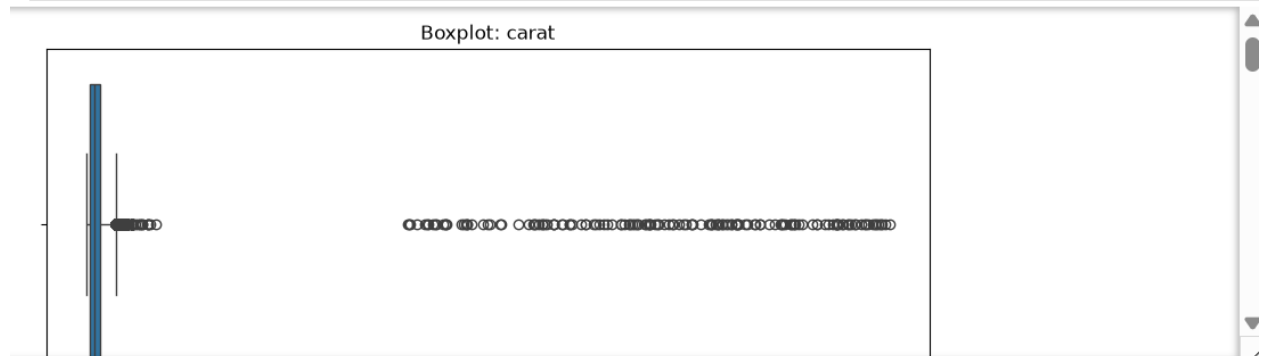


Рис. 19 – Удаление строк с пропусками через .dropna() и визуализация выбросов с помощью boxplot для числовых признаков датасета diamonds.

```
def remove_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[column] >= lower) & (data[column] <= upper)]

for col in ["carat", "depth", "table", "x", "y", "z"]:
    df = remove_outliers_iqr(df, col)

df.head()
```

	карат	вырезать	Цвет	ясность	глубина	таблица	x	y	z
0	1.50	Справедливый	G	SI1	64.5	57.0	7.15	7.09	4.59
1	0.70	Идеальный вариант	E	ПРОТИВ2	61.4	57.0	5.72	5.78	3.53
2	1.22	Премиум	G	ПРОТИВ1	61.3	58.0	6.91	6.89	4.23
3	0.51	Премиум	E	ПРОТИВ2	62.5	60.0	5.08	5.10	3.18
4	2.02	Очень Хорошо	J	SI2	59.2	60.0	8.27	8.39	4.91

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

df.head()
```

	карат	вырезать	Цвет	ясность	глубина	таблица	x	y	z
0	1.776201	Справедливый	G	SI1	2.447919	-0.141333	1.432148	1.378806	1.695091
1	-0.127819	Идеальный вариант	E	ПРОТИВ2	-0.345564	-0.141333	0.074730	0.128209	0.066886
2	1.109794	Премиум	G	ПРОТИВ1	-0.435676	0.352009	1.204330	1.187875	1.142116
3	-0.580024	Премиум	E	ПРОТИВ2	0.645672	1.338693	-0.532787	-0.520956	-0.470728
4	3.013814	Очень Хорошо	J	SI2	-2.328036	1.338693	2.495301	2.619857	2.186624

Рис. 20 - Удаление выбросов с помощью IQR и стандартизация числовых признаков с использованием StandardScaler в датасете diamonds.

```
|: cut_mapping = {
    "Fair": 0, "Good": 1, "Very Good": 2, "Premium": 3, "Ideal": 4
}

df["cut"] = df["cut"].map(cut_mapping)

clarity_mapping = {
    "I1": 0, "SI2": 1, "SI1": 2, "VS2": 3,
    "VS1": 4, "VVS2": 5, "VVS1": 6, "IF": 7
}

df["clarity"] = df["clarity"].map(clarity_mapping)

df.head()
```

```
|:      карат  вырезать  Цвет  ясность  глубина  таблица      x      y      z
0  1.776201      0.0    G      2  2.447919 -0.141333  1.432148  1.378806  1.695091
1 -0.127819      4.0    E      3 -0.345564 -0.141333  0.074730  0.128209  0.066886
2  1.109794      3.0    G      4 -0.435676  0.352009  1.204330  1.187875  1.142116
3 -0.580024      3.0    E      3  0.645672  1.338693 -0.532787 -0.520956 -0.470728
4  3.013814      2.0    J      1 -2.328036  1.338693  2.495301  2.619857  2.186624
```

```
|: df = pd.get_dummies(df, columns=["color"], drop_first=True)
print(df.head())

Таблица глубины пропила в каратах x y z \
0 1.776201 0.0 2 2.447919 -0.141333 1.432148 1.378806 1.695091
1 -0.127819 4.0 3 -0.345564 -0.141333 0.074730 0.128209 0.066886
2 1.109794 3.0 4 -0.435676 0.352009 1.204330 1.187875 1.142116
3 -0.580024 3.0 3 0.645672 1.338693 -0.532787 -0.520956 -0.470728
4 3.013814 2.0 1 -2.328036 1.338693 2.495301 2.619857 2.186624

color_E color_F color_G color_H color_I color_J
0 False False True False False False False
1 True False False False False False False
2 False False True False False False False
3 Верно Неверно Неверно Неверно Неверно
4 Неверно Неверно Неверно Неверно Верно
```

Рис. 21 – Кодирование порядковых признаков cut и clarity с помощью .map(), а также one-hot кодирование номинального признака color через pd.get_dummies() с исключением дамми-ловушки (drop_first=True).

Задания:

Если используется файл общеизвестного датасета, он должен находиться в папке `data` репозитория.

Задание 1. Обнаружение и обработка пропущенных значений

Датасет: `titanic` (пассажиры Титаника)

Источник: `seaborn.load_dataset("titanic")`

Инструкции:

1. Загрузите датасет `titanic`.
2. Определите количество пропущенных значений в каждом столбце.
3. Визуализируйте пропуски с помощью библиотеки `missingno`.
4. Заполните пропущенные значения:
 - признак `age` — средним значением;
 - признак `embarked` — наиболее частым значением;
 - признак `deck` — удалите.
5. Отобразите информацию о таблице до и после обработки (`.info()` , `.isna().sum()`).

```

: #Задание 1. Titanic – пропуски

import pandas as pd
import numpy as np
import missingno as msno
import matplotlib.pyplot as plt
import seaborn as sns

df = sns.load_dataset("titanic")
print(df.info())
print(df.isna().sum())

msno.matrix(df)
plt.show()

df["age"] = df["age"].fillna(df["age"].mean())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df = df.drop(columns=["deck"])

print(df.info())
print(df.isna().sum())

<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    str
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    str
8   class       891 non-null    category
9   who         891 non-null    str
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    str
13  alive       891 non-null    str
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 100.4 KB
None
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64

```

Рис. 22 — код к заданию 1

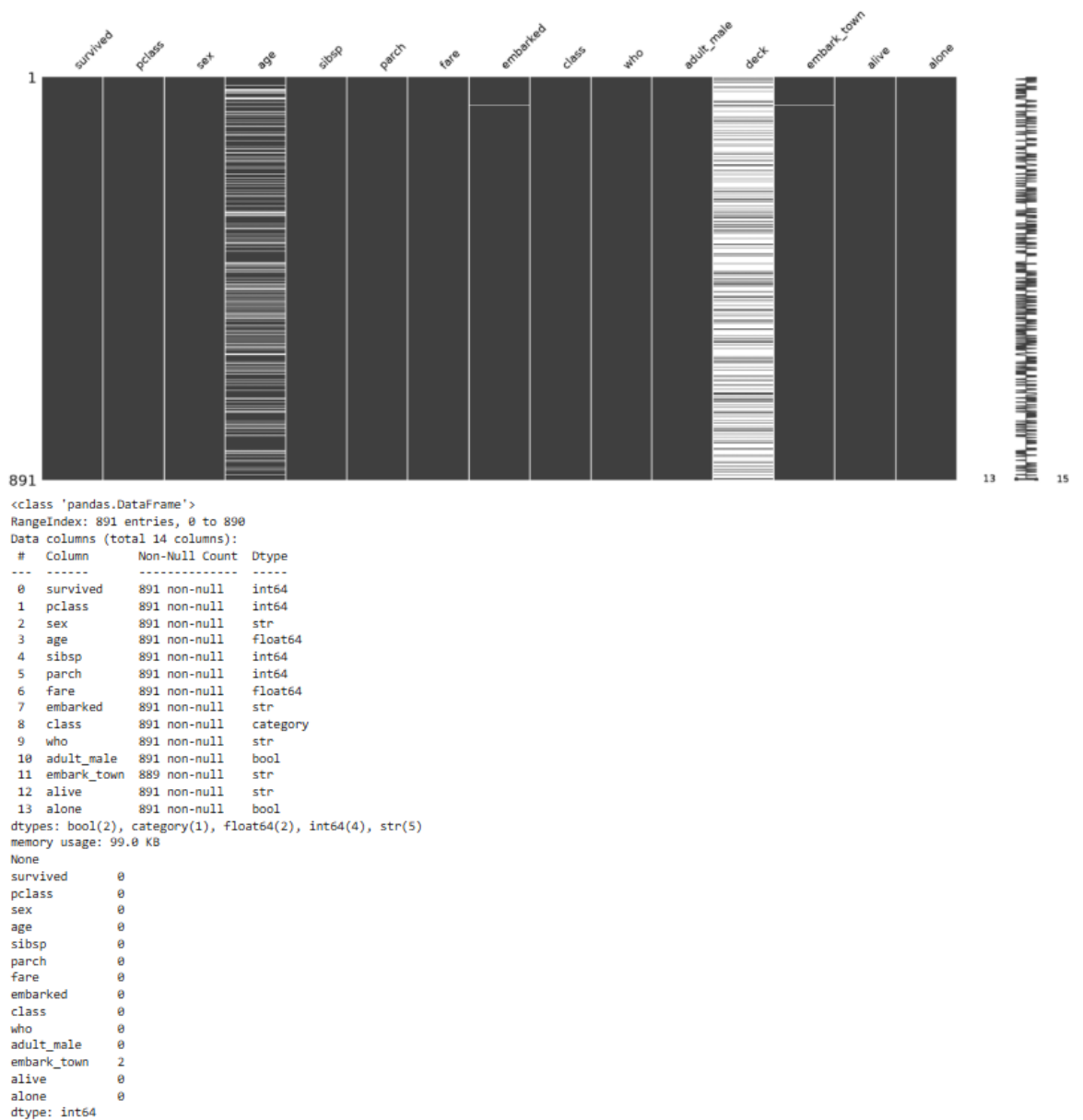


Рис. 23 – код к заданию 1

Задание 2. Обнаружение и удаление выбросов

Датасет: `penguins` (описание антарктических пингвинов)

Источник: `seaborn.load_dataset("penguins")`

Инструкции:

1. Загрузите датасет `penguins`.
2. Постройте boxplot-графики для признаков `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`.
3. Используя метод межквартильного размаха (IQR), выявите и удалите выбросы по каждому из указанных признаков.
4. Сравните размеры датасета до и после фильтрации.
5. Постройте boxplot-график до и после удаления выбросов для одного из признаков.

```
[18]: #Задание 2. Penguins – выбросы
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("penguins")
print("До:", len(df))

cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]

for col in cols:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[col])
    plt.title(f"До: {col}")
    plt.show()

def remove_outliers(data, col):
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[col] >= lower) & (data[col] <= upper)]

for col in cols:
    df = remove_outliers(df, col)

print("После:", len(df))

for col in cols:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[col])
    plt.title(f"После: {col}")
    plt.show()
```

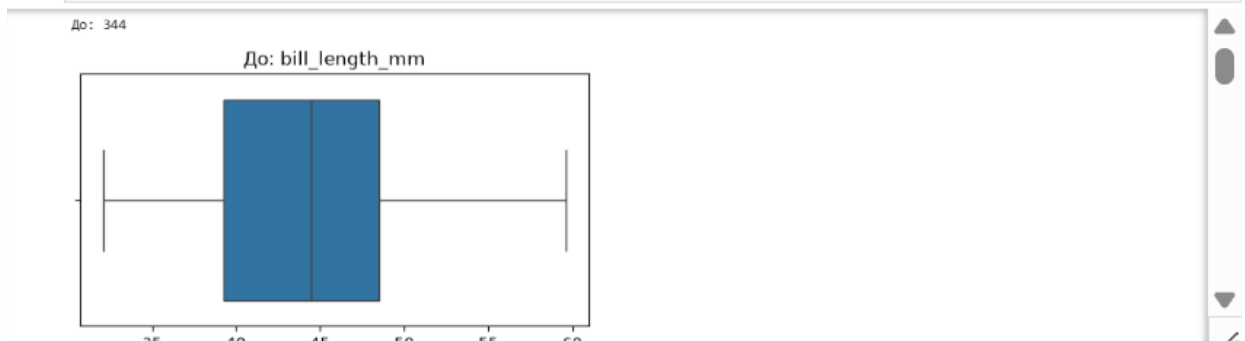


Рис. 24 – код к заданию 2

Задание 3. Масштабирование числовых признаков

Датасет: california housing

Источник: `from sklearn.datasets import fetch_california_housing`

Инструкции:

1. Загрузите данные с помощью `fetch_california_housing(as_frame=True)`.
2. Преобразуйте данные в `pandas.DataFrame`.
3. Выполните:
 - стандартизацию признаков с помощью `StandardScaler`;
 - нормализацию в диапазон `[0, 1]` с помощью `MinMaxScaler` (на копии таблицы).
4. Постройте гистограммы распределения признака `MedInc` до и после масштабирования.
5. Сравните поведение шкал на гистограммах.

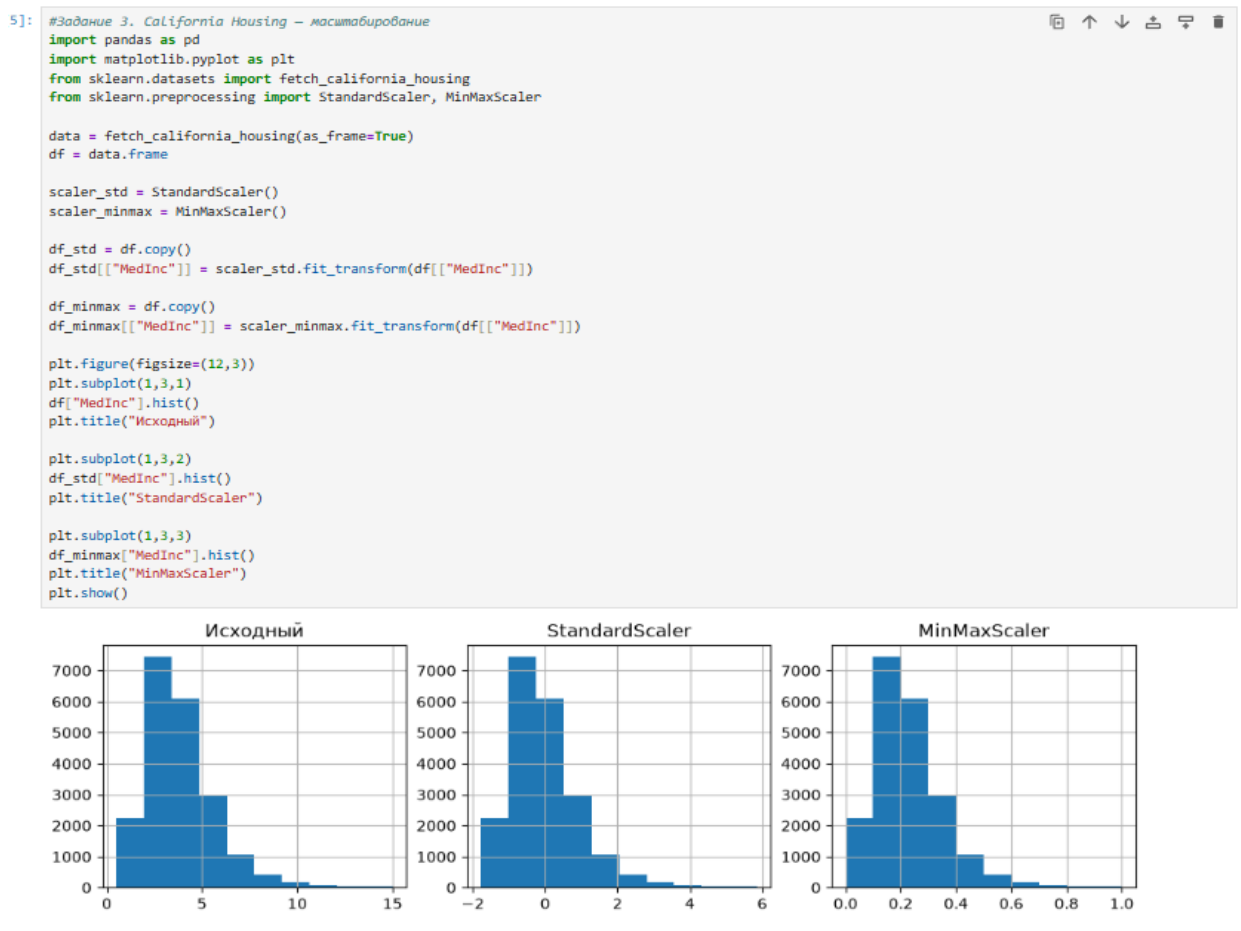


Рис. 25 – код к заданию 3

Задание 4. Кодирование категориальных признаков

Датасет: `adult` (перепись населения США, income dataset)

Источник: <https://archive.ics.uci.edu/ml/datasets/adult>

Или через библиотеку `sklearn.datasets.fetch_openml("adult")`

Инструкции:

1. Загрузите данные и отберите признаки:
 - категориальные: `education`, `marital-status`, `occupation`;
 - целевой признак: `income`.
2. Проведите Label Encoding для признака `education`, предполагая, что уровни образования упорядочены.
3. Примените One-Hot Encoding к признакам `marital-status` и `occupation`.
4. Проверьте итоговую размерность таблицы до и после кодирования.
5. Убедитесь, что в one-hot-кодировании не присутствует дамми-ловушка.

```

: #Задание 4. Adult – кодирование
import pandas as pd
from sklearn.preprocessing import LabelEncoder

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
cols = ["age", "workclass", "fnlwgt", "education", "education-num", "marital-status",
        "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss",
        "hours-per-week", "native-country", "income"]
df = pd.read_csv(url, header=None, names=cols)
df = df[["education", "marital-status", "occupation", "income"]]
print("До:", df.shape)

education_order = [
    " Preschool":0, " 1st-4th":1, " 5th-6th":2, " 7th-8th":3, " 9th":4,
    " 10th":5, " 11th":6, " 12th":7, " HS-grad":8, " Some-college":9,
    " Assoc-voc":10, " Assoc-acdm":11, " Bachelors":12, " Masters":13,
    " Prof-school":14, " Doctorate":15
]
df["education_encoded"] = df["education"].map(education_order)
df = pd.get_dummies(df, columns=["marital-status", "occupation"], drop_first=True)
print("После:", df.shape)
print(df.head())

До: (32561, 4)
После: (32561, 23)
   education  income  education_encoded  marital-status_Married-AF-spouse \
0  Bachelors  <=50K                12                                False
1  Bachelors  <=50K                12                                False
2    HS-grad  <=50K                 8                                False
3     11th    <=50K                 6                                False
4  Bachelors  <=50K                12                                False

   marital-status_Married-civ-spouse  marital-status_Married-spouse-absent \
0                                False                                False
1                                True                                 False
2                                False                                False
3                                True                                 False
4                                True                                 False

   marital-status_Never-married  marital-status_Separated \
0                                True                        False
1                                False                       False
2                                False                       False
3                                False                       False
4                                False                       False

   marital-status_Widowed  occupation_Adm-clerical  ... \
0                        False                      True  ...
1                        False                      False  ...
2                        False                      False  ...
3                        False                      False  ...
4                        False                      False  ...

   occupation_Farming-fishing  occupation_Handlers-cleaners \
0                        False                      False
1                        False                      False
2                        False                      True
3                        False                      True
4                        False                      False

   occupation_Machine-op-inspct  occupation_Other-service \
0                        False                      False
1                        False                      False
2                        False                      False
3                        False                      False
4                        False                      False

   occupation_Priv-house-serv  occupation_Prof-specialty \
0                        False                      False
1                        False                      False
2                        False                      False
3                        False                      False
4                        False                      True

   occupation_Protective-serv  occupation_Sales  occupation_Tech-support \
0                        False                      False                      False
1                        False                      False                      False
2                        False                      False                      False
3                        False                      False                      False
4                        False                      False                      False

   occupation_Transport-moving
0                        False
1                        False
2                        False
3                        False
4                        False

```

Рис. 26 – код к заданию 4

Задание 5. Комплексный EDA

Датасет: heart disease (заболевания сердца)

Источник: <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>

(Загрузите вручную или запросите файл у преподавателя)

Инструкции: Выполните полный цикл EDA:

1. Обзор структуры данных (`.info()` , `.describe()`).
2. Обнаружение и обработка пропущенных значений.
3. Обнаружение и удаление выбросов по признакам: `age` , `cholesterol` , `restingbp` , `maxhr` .
4. Масштабирование числовых признаков.
5. Кодирование категориальных признаков: `sex` , `chestpain` , `exerciseangina` , `restecg` .
6. Подготовьте отчёт в виде Jupyter-ноутбука с комментариями к каждому этапу и промежуточными результатами.

```
[6]: #Задание 5. Heart Disease – комплексный EDA

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target

print(df.info())
print(df.isna().sum())

cols = ["mean radius", "mean texture", "mean perimeter", "mean area"]
for col in cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower) & (df[col] <= upper)]

num_cols = ["mean radius", "mean texture", "mean perimeter", "mean area"]
scaler = StandardScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])

df["target"] = LabelEncoder().fit_transform(df["target"])

print("Готово:", df.shape)
print(df.head())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   mean radius            569 non-null   float64
1   mean texture           569 non-null   float64
2   mean perimeter         569 non-null   float64
3   mean area              569 non-null   float64
4   mean smoothness        569 non-null   float64
5   mean compactness       569 non-null   float64
6   mean concavity         569 non-null   float64
7   mean concave points    569 non-null   float64
8   mean symmetry          569 non-null   float64
9   mean fractal dimension 569 non-null   float64
10  radius error           569 non-null   float64
11  texture error          569 non-null   float64
12  perimeter error        569 non-null   float64
```

Рис. 27 – код к заданию 5

Индивидуальные задания:

Выполните последовательные шаги исследовательского анализа:

1. Обзор структуры данных

- Загрузите датасет.
- Выведите общую информацию (`.info()` , `.describe()`).
- Опишите: сколько признаков, каких типов, какова структура целевого признака.

```
# 1. Обзор структуры данных
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder

df = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data",
                 header=None,
                 names=["age", "workclass", "fnlwt", "education", "education-num",
                      "marital-status", "occupation", "relationship", "race", "sex",
                      "capital-gain", "capital-loss", "hours-per-week", "native-country", "income"])

print("Признаки:", df.columns.tolist())
print("\nИнформация:")
df.info()
print("\nСтатистика:")
print(df.describe())

Признаки: ['возраст', 'рабочий класс', 'НОСРГТ', 'образование', 'количество образований', 'семейное положение', 'род занятий', 'отношения', 'раса',
'пол', 'прирост капитала', 'потеря капитала', 'количество часов в неделю', 'страна происхождения', 'доход']

Информация:
<класс 'pandas.DataFrame'>
RangeIndex: 32561 запись, от 0 до 32560
Столбцы данных (всего 15 столбцов):
# Столбец Количество ненулевых значений Тип данных
-----
0 age 32561 ненулевых значений int64
1 workclass 32561 ненулевых значений str
2 fnlwt 32561 ненулевой int64
3 education 32561 ненулевой str
4 education-num 32561 ненулевой int64
5 семейное положение 32561 ненулевая улица
6 профессия 32561 ненулевая улица
7 родство 32561 непустая строка
8 раса 32561 непустая строка
9 пол 32561 непустая строка
10 прирост капитала 32561 непустая строка int64
11 убыток капитала 32561 непустая строка int64
12 количество часов в неделю 32561 непустая строка int64
13 ненулевых данных о стране происхождения 32561
14 ненулевых данных о доходах 32561
типы данных: int64 (6), str (9)
объем используемой памяти: 6,5 МБ

Статистика:
возраст, образование-num капитал-прирост капитала-потеря \
count 32561.000000 3.256100e+04 32561.000000 32561.000000 32561.000000
mean 38.581647 1.897784e+05 10.080679 1077.648844 87.303830
std 13.640433 1.055500e+05 2.572720 7385.292085 402.960219
min 17.000000 1.228500e+04 1.000000 0.000000 0.000000
25% 28.000000 1.178270e+05 9.000000 0.000000 0.000000
50% 37.000000 1.783560e+05 10.000000 0.000000 0.000000
75% 48.000000 2.370510e+05 12.000000 0.000000 0.000000
max 90.000000 1.484705e+06 16.000000 99999.000000 4356.000000

количество часов в неделю
количество 32561.000000
среднее значение 40,437456
стандартное отклонение 12,347429
минимальное значение 1.000000
25 % 40.000000
50 % 40.000000
75 % 45.000000
максимум 99,000000
```

Рис. 28 – код к инд. заданию 1

2. ОБНАРУЖЕНИЕ И ОБРАБОТКА ПРОПУСКОВ

- Определите, есть ли пропущенные значения.
- Обоснуйте выбранный способ их устранения (удаление, заполнение средним/модой и т.д.).
- Примените выбранный способ.

```
# 2. Обнаружение и обработка пропусков
print("Пропуски:\n", df.isna().sum())
```

```
Пропуски:
MedInc 0
Возраст дома 0
Среднее количество комнат 0
Среднее количество спален 0
Население 0
Среднее количество жильцов 0
Широта 0
Долгота 0
Средняя стоимость дома 0
dtype: int64
```

Рис. 29 – код к инд. заданию 2

3. ОБНАРУЖЕНИЕ И УДАЛЕНИЕ ВЫБРОСОВ

- Выберите 3–5 числовых признаков.
- Используя метод IQR, удалите выбросы.
- Сравните объём данных до и после очистки.

```
# 3. Обнаружение и удаление выбросов (IQR)
```

```
def remove_outliers(data, col):
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[col] >= lower) & (data[col] <= upper)]

print("До:", len(df))
for col in ["MedInc", "AveRooms", "AveBedrms", "Population", "AveOccup"]:
    df = remove_outliers(df, col)
print("После:", len(df))
```

```
До: 20640
После: 16813
```

Рис. 30 – код к инд. заданию 3

4. МАСШТАБИРОВАНИЕ ЧИСЛОВЫХ ПРИЗНАКОВ

- Выполните стандартизацию (z-преобразование) с помощью `StandardScaler`.
- Объясните, зачем выполняется масштабирование.

```
# 4. Масштабирование числовых признаков
num_cols = df.select_dtypes(include=[np.number]).columns
scaler = StandardScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])
print("Масштабирование выполнено")
```

Масштабирование выполнено

Рис. 31 – код к инд. заданию 4

5. КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ

- Выполните:
 - **Label Encoding** для порядковых признаков (при наличии);
 - **One-Hot Encoding** для номинальных признаков.
- Проверьте, исключена ли дамми-ловушка.

```
# 5. Кодирование категориальных признаков
# В датасете нет категориальных, создадим искусственный для примера
df["Ocean_Proxy"] = np.random.choice(["Near", "Far", "Medium"], size=len(df))

# Label Encoding (порядковый)
le = LabelEncoder()
df["Ocean_Proxy_Label"] = le.fit_transform(df["Ocean_Proxy"])

# One-Hot Encoding (номинальный)
df = pd.get_dummies(df, columns=["Ocean_Proxy"], drop_first=True)

print("Кодирование выполнено")
print(df.head())
```

Кодирование выполнено

```
MedInc HouseAge AveRooms AveBedrms Население AveOccup Latitude \
3 1.306354 1.822621 0.622013 0.389206 -1.128897 -0.472753 1.030153
4 0.091339 1.822621 1.065587 0.511007 -1.117819 -1.052713 1.030153
5 0.220217 1.822621 -0.386119 0.853342 -1.358373 -1.118500 1.030153
6 -0.035173 1.822621 -0.223541 -1.458639 -0.280630 -1.136686 1.025462
7 -0.399698 1.822621 -0.351866 0.218606 -0.180927 -1.674984 1.025462
```

```
Средняя долгота дома Ocean_Proxy_Label Ocean_Proxy_Medium \
3 -1.31572 1.299550 0 False
4 -1.31572 1.307959 1 Верно
5 -1.31572 0.630573 1 Верно
6 -1.31572 0.906199 1 Верно
7 -1.31572 0.366159 1 Верно
```

```
Ocean_Proxy_Near
3 Ложно
4 Ложно
5 Неверно
6 Неверно
7 Неверно
```

Рис. 32 – код к инд. заданию 5

6. ФИНАЛЬНЫЙ НАБОР ДАННЫХ

- Убедитесь, что датасет не содержит пропусков, выбросов, категориальных данных в строковом виде.
- Признаки приведены к числовому виду, масштабированы.
- Представьте итоговый `DataFrame`, готовый к использованию в моделях.

```
# 6. Финальный набор данных
```

```
print("Итоговый DataFrame:")
```

```
print(df.info())
```

```
print("\nПервые 5 строк:")
```

```
print(df.head())
```

Итоговый DataFrame:

```
<class 'pandas.DataFrame'>
```

Индекс: 16813 записей, от 3 до 20639

Столбцы данных (всего 12 столбцов):

Столбец Количество ненулевых значений Тип данных

0 MedInc 16813 ненулевых значений float64

1 HouseAge 16813 ненулевых значений float64

2 AveRooms 16813 непустое значение типа float64

3 AveBedrms 16813 непустое значение типа float64

4 Population 16813 непустое значение типа float64

5 AveOccup 16813 непустое значение типа float64

6 Latitude 16813 непустое значение типа float64

7 Longitude 16813 непустое значение типа float64

8 MedHouseVal 16813 непустое значение типа float64

9 Ocean_Proxy_Label 16813 непустое значение типа int64

10 Ocean_Proxy_Medium 16813 непустое значение типа bool

11 Ocean_Proxy_Near 16813 непустое значение типа bool

типы данных: bool(2), float64(9), int64(1)

использование памяти: 1,4 МБ

Нет

Первые 5 строк:

```
MedInc HouseAge AveRooms AveBedrms Население AveOccup Latitude \
```

```
3 1.306354 1.822621 0.622013 0.389206 -1.128897 -0.472753 1.030153
```

```
4 0,091339 1,822621 1,065587 0,511007 -1,117819 -1,052713 1,030153
```

```
5 0,220217 1,822621 -0,386119 0,853342 -1,358373 -1,118500 1,030153
```

```
6 -0,035173 1,822621 -0,223541 -1,458639 -0,280630 -1,136686 1,025462
```

```
7 -0,399698 1.822621 -0.351866 0.218606 -0.180927 -1.674984 1.025462
```

```
Средняя долгота дома Ocean_Proxy_Label Ocean_Proxy_Medium \
```

```
3 -1.31572 1.299550 0 False
```

```
4 -1.31572 1.307959 1 Верно
```

```
5 -1.31572 0.630573 1 Верно
```

```
6 -1.31572 0.906199 1 Верно
```

```
7 -1.31572 0.366159 1 Верно
```

```
Ocean_Proxy_Near
```

```
3 Ложно
```

```
4 Ложно
```

```
5. Неверно
```

```
6. Неверно
```

```
7. Неверно
```

Рис. 33 – код к инд. заданию 6

Ответы на контрольные вопросы:

1. Какие типы проблем могут возникнуть из-за пропущенных значений в данных?

Искажение статистик (среднее, дисперсия), ошибки в моделях, потеря данных при удалении.

2. Как с помощью методов pandas определить наличие пропущенных значений?

`.isna()` или `.isnull()` — показывают True для NaN, `.sum()` — считает количество.

3. Что делает метод `.dropna()` и какие параметры он принимает? Удаляет строки/столбцы с NaN. Параметры: `axis` (0 — строки, 1 — столбцы), `how` ('any'/'all'), `thresh` (мин. непустых), `subset`.

4. Чем различаются подходы заполнения пропусков средним, медианой и модой?

Среднее — для симметричных данных. Медиана — устойчива к выбросам. Мода — для категориальных признаков.

5. Как работает метод `fillna(method='ffill')` и в каких случаях он применим? Заполняет NaN предыдущим значением (forward fill). Применим для временных рядов и данных с порядком.

6. Какую задачу решает метод `interpolate()` и чем он отличается от `fillna()`? Интерполяция восстанавливает пропуски по тренду (линейно, полиномиально). `fillna()` — просто вставляет значения.

7. Что такое выбросы и почему они могут исказить результаты анализа? Выбросы — аномальные значения. Искажают среднее, дисперсию, коэффициенты моделей и визуализации.

8. В чём суть метода межквартильного размаха (IQR) и как он используется для обнаружения выбросов?

$IQR = Q3 - Q1$. Выбросы — значения за пределами $[Q1 - 1.5IQR, Q3 + 1.5IQR]$.

9. Как вычислить границы IQR и применить их в фильтрации? $lower = Q1 - 1.5 * IQR$, $upper = Q3 + 1.5 * IQR$. Фильтр: `df[(df[col] >= lower) & (df[col] <= upper)]`.

10. Что делает метод `.clip()` и как его можно использовать для обработки выбросов?

Обрезает значения до указанных границ. `df[col].clip(lower, upper)` заменяет выбросы на границы.

11. Зачем может потребоваться логарифмическое преобразование числовых признаков?

Сжимает большие значения, делает распределение более нормальным, уменьшает влияние выбросов.

12. Какие графические методы позволяют обнаружить выбросы (указать не менее двух)?

Boxplot (ящик с усами), гистограмма, scatterplot (точечный график).

13. Почему важно быть осторожным при удалении выбросов из обучающих данных?

Выбросы могут быть ценными данными (редкие события). Удаление может привести к потере информации и ухудшению модели.

14. Зачем необходимо масштабирование признаков перед обучением моделей? Чтобы признаки были в едином масштабе, и модель не отдавала предпочтение признакам с большими значениями.

15. Чем отличается стандартизация от нормализации? Стандартизация: среднее=0, std=1. Нормализация: значения в диапазоне [0, 1].

16. Что делает StandardScaler и как рассчитываются преобразованные значения?

Вычитает среднее и делит на std: $(x - \text{mean}) / \text{std}$. Приводит к среднему 0 и std 1.

17. Как работает MinMaxScaler и когда его использование предпочтительно? $(x - \text{min}) / (\text{max} - \text{min}) \rightarrow [0, 1]$. Используется, когда нужны положительные значения и сохранение пропорций.

18. В чём преимущества RobustScaler при наличии выбросов? Использует медиану и IQR вместо среднего и std — устойчив к выбросам.

19. Как реализовать стандартизацию с помощью .mean() и .std() вручную в pandas?

$(df[\text{col}] - df[\text{col}].\text{mean}()) / df[\text{col}].\text{std}()$

20. Какие типы моделей наиболее чувствительны к масштабу признаков? Линейные модели, SVM, нейронные сети, методы на основе расстояний (KNN, кластеризация).

21. Почему необходимо преобразовывать категориальные признаки перед обучением модели?

Модели работают только с числами. Категории нужно перевести в числовой формат.

22. Что такое порядковый признак? Приведите пример. Есть логический порядок. Пример: образование (начальное < среднее < высшее).

23. Что такое номинальный признак? Приведите пример. Нет порядка. Пример: город (Москва, СПб, Казань).
24. Как работает метод `.factorize()` и для каких случаев он подходит? Кодировать категории числами в порядке появления. Подходит для быстрого кодирования без задания порядка.
25. Как применить метод `.map()` для кодирования категориальных признаков с известным порядком? Создать словарь {"категория": число} и применить: `df[col].map(mapping)`.
26. Что делает класс `OrdinalEncoder` из `scikit-learn`? Кодировать порядковые признаки числами в заданном порядке. Поддерживает работу в `Pipeline`.
27. В чём суть `one-hot` кодирования и когда оно применяется? Создаёт отдельный бинарный столбец для каждой категории. Применяется для номинальных признаков.
28. Как избежать дамми-ловушки при `one-hot` кодировании? Исключить один столбец: `drop_first=True` в `pd.get_dummies()` или `drop='first'` в `OneHotEncoder`.
29. Как работает `OneHotEncoder` из `scikit-learn` и чем он отличается от `pd.get_dummies()`? `OneHotEncoder` — для работы в `Pipeline`, поддерживает разреженные матрицы, можно задать `drop`. `pd.get_dummies()` — проще, но не для `Pipeline`.
30. В чём суть метода `target encoding` и какие риски он в себе несёт? Заменяет категорию на среднее целевой переменной. Риски: утечка данных и переобучение.

Вывод: В ходе работы освоены основные этапы EDA: обработка пропусков, удаление выбросов, масштабирование и кодирование признаков. Данные приведены к чистому числовому виду, готовому для машинного обучения.